Implementation Document

for

Travelly

Version 1.0

Prepared by

Group #17 — **Group Name: 10 Bit**

Ayush Jha	230258	ayushj23@iitk.ac.in
Khushi Khandelwal	230561	khushik23@iitk.ac.in
Pranjali Singh	220796	pranjalis22@iitk.ac.in
Mohd Sarim Ali	230657	sarimali23@iitk.ac.in
Raghav Madan	220853	raghavm22@iitk.ac.in
Rigved Koli	230855	brigved23@iitk.ac.in
Ronit Kumar	230875	ronitk23@iitk.ac.in
Rushabh Pandya	230732	rushabhbp23@iitk.ac.in
Sanket Agarwal	230920	sanketag23@iitk.ac.in
Tanmay Roy	231082	tanmayroy23@iitk.ac.in

Course: CS253

Mentor TA: *V Arvind*

Date: 27/03/2026

Table of Contents

Contents II

Revisions II

1 IMPLEMENTATION DETAILS 1

2 CODEBASE 2

3 COMPLETENESS 3

Appendix A - Group Log 4

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
1.0	Ayush Jha Khushi Khandelwal Pranjali Singh Mohd Sarim Ali Raghav Madan Rigved Koli Ronit Kumar Rushabh Pandya Sanket Agarwal Tanmay Roy	First draft of Implementation Document	27/03/26

1 Implementation Details

Programming Languages

The Travelly application was implemented using two primary programming languages. The frontend was developed in **Dart**, while the backend was implemented in **JavaScript** running on Node.js. Dart was selected because it is the native language for Flutter and enables efficient development of a high-quality cross-platform user interface from a single codebase. JavaScript was chosen for the backend because of its strong ecosystem for REST API development and its absolute compatibility with lightweight, modular server-side frameworks such as Express.js. This event-driven architecture heavily optimizes I/O-bound requests typical of our application, including parsing external API JSON payloads and processing concurrent database writes for group expenses.

Frontend Framework

The frontend was developed using the **Flutter** framework. Flutter was chosen because it supports cross-platform application development through a single codebase, thereby reducing development time and maintenance effort. Its widget-based architecture made it easier to design and manage multiple complex dynamic user interfaces such as authentication, multi-person expense ledgers, settlements, document management, gallery views, and graphical route planning. Compared to building separate Android and iOS applications natively, Flutter offers higher development efficiency and better consistency in design and behavior across platforms.

Backend Framework

The backend was implemented using **Express.js** on top of Node.js utilizing a strictly decoupled Model-Route-Controller-Service (MRCS) architectural pattern. Express.js was chosen because it is lightweight, flexible, and well-suited for building RESTful APIs. It allowed the project to be organized functionally, where lightweight controllers parse HTTP arguments and immediately defer heavy logic to specific services (e.g., deferring computational split math directly to `expenseService.js`). Compared to heavier backend frameworks like Java Spring Boot, Express.js introduces less boilerplate, allowing faster end-to-end development without sacrificing the capacity to handle rigorous execution logic (like debt algorithms and remote mapping calculations).

Database System

The project uses **PostgreSQL** as its primary database system. PostgreSQL was selected because it is a reliable relational database with robust support for strict structured data, transactions, and complex relations. Because Travelly fundamentally relies on strict foreign-key integrity (e.g., an `Expense` cannot exist without a valid `Group`, and a `Settlement` securely maps exact User relations), a relational database was far more appropriate than NoSQL alternatives like MongoDB. PostgreSQL also provides necessary ACID compliance specifically guaranteeing that real-time peer-to-peer monetary balances cannot be left in corrupted states.

ORM and Database Access

For backend database interaction, **Prisma ORM** was used. Prisma simplifies database access, schema definition, and query handling while reducing the amount of handwritten SQL required, significantly mitigating SQL injection vulnerabilities natively. Its schema-driven approach generates explicit localized TypeScript/JavaScript type definitions based directly on our `schema.prisma` file, ensuring strict consistency between application models and the actual relational database structure.

Authentication and Identity Management

The application uses **Firebase Authentication** for user authentication, heavily delegating secure Identity provisioning horizontally. The backend uses the Firebase Admin SDK for actively verifying Firebase ID tokens. Firebase Authentication was selected because it provides a secure, ready-made identity system mapping seamlessly to stateless JWTs (JSON Web Tokens). The frontend intercepts outbound HTTP requests using the `dio` package, injecting this JWT securely into the `Authorization` header. Our backend middleware (`authMiddleware.js`) mathematically intercepts the payload, cryptographically validating the signature against Firebase before permitting any interaction with closed-loop group environments.

Algorithmic Implementations (Custom Business Logic)

Moving beyond standard generic architectures, Travelly incorporates specific computational problem-solving mechanisms:

1. **Algorithmic Debt Simplification (`settlementService.js`)**: Realizing the chaos of multi-member group trips, Travelly includes a configurable "Simplify Debts" system. When executed, a mathematical greedy algorithm iteratively sorts active creditors against active debtors by simple magnitude. It actively pairs the maximum creditor to the maximum debtor, directly severing cyclic/redundant relationships and drastically minimizing the total count of required peer-to-peer settlement transactions.
2. **Geospatial Route Optimization (`orsService.js`)**: To intelligently organize trip itineraries, a localized server-side algorithm maps user-defined coordinates applying Haversine distance heuristics to execute a "nearest-neighbor" sequence calculation locally before pushing refined metrics safely to remote mapping tools.

Frontend Libraries

Several Flutter libraries were carefully integrated to solve specific frontend functionalities:

- **provider**: For state management. It centrally stores logic like user session data or actively viewed group arrays to avoid heavy "prop drilling" down the widget tree.
- **http and dio**: For network communication and secure file transfer mechanisms, utilizing interceptors for API token injection.
- **file_picker and image_picker**: For selecting structured PDF documents, localized images, and media natively across iOS/Android filesystems.

- **cached_network_image**: For efficient image rendering and persistent memory caching, drastically reducing repeated network latency on group galleries.
- **flutter_cached_pdfview**: For natively previewing heavy PDF itineraries/tickets seamlessly inside the app scope.
- **shared_preferences**: For fast, lightweight dictionary storage for maintaining user UI scopes and session keys locally.
- **flutter_dotenv**: For securely injecting environment-specific network configuration values globally.
- **google_fonts and google_nav_bar**: For unified premium UI styling and standardized navigational structures.
- **url_launcher**: For spawning external intents, securely generating UPI deep links externally for rapid app-switching transactions.

Backend Libraries

The backend uses several explicit Node.js packages orchestrating functionality independent of simple Express.js routing:

- **cors**: Safely dictating cross-origin resource sharing between our mobile execution domains and raw endpoints.
- **helmet**: Standardizing the deployment of essential HTTP header guards strictly enhancing API security natively.
- **morgan**: Handling active physical console request logging actively mapping latency benchmarks during runtime testing.
- **cookie-parser**: Native extraction logic for explicitly handled functional web sessions.
- **multer**: Directly capturing active multipart/form-data HTTP payloads, acting as an ingestion buffer successfully executing the direct system write commands depositing PDFs and JPEGs strictly to `uploads/groups/:id/` filesystem destinations.
- **axios**: Operating server-to-server functional requests, largely pushing mapping contexts successfully to the remote OpenRouteService APIs.
- **geoip-lite**: Resolving connected client IP addresses strictly calculating the correct default localization currency logic without requesting explicit user input formats natively.
- **express-validator**: Systematically enforcing payload bounds avoiding server crashing on explicit bad data formats during routing interactions.
- **jest**: Establishing robust localized scripting workflows handling explicit logic validation actively testing computational calculations successfully disconnected from active servers.

Media and File Storage

Uploaded files, such as group profile photos, gallery media, and functional documents, are explicitly intercepted via `multer` and strictly bound to local subdirectories matching their `groupId` on the server filesystem. Conversely, their respective physical localized URIs, original encoded byte-sizes, and MIME Types form explicit semantic pointers generated cleanly into PostgreSQL. This mechanism avoids injecting physically large binary objects into active database indices protecting server performance, while explicitly retaining structured lifecycle tracking for efficient deletion handling seamlessly across our models.

Build and Development Tooling

The frontend leverages the standard Flutter SDK build pipeline utilizing `flutter pub` resolving native structural dependencies natively orchestrating Dart compilation effectively parsing generic UI elements down onto raw C++ system hooks cleanly translating for both Android and iOS targets natively. Concurrently, the Backend strictly organizes modules applying `npm` tracking strictly alongside `nodemon` rapidly tracking local source variations executing logical hot-restarts dramatically quickening isolated development speeds securely alongside the Prisma CLI handling rigid structural schema migration configurations efficiently.

2 Codebase

The Travelly project is logically separated into two distinct repositories to strictly maintain the separation of concerns between our client-side visualizations and server-side data manipulations.

1. **Frontend**: Developed using the Dart-based Flutter framework, focused entirely on the user interface, routing logic, and device-native capabilities (like accessing the local file picker or camera).
2. **Backend**: Developed using the Express.js framework on top of Node.js, focused heavily on secure database interactions, computationally intensive tasks (like route optimization and debt simplification), and secure API provisioning.

The source code for both environments is hosted collaboratively via an organization set up on GitHub.
Github Repository: https://github.com/CS253

2.1 Backend Architecture

The backend repository applies a robust, heavily decoupled **Model-Route-Controller-Service** architectural pattern. This ensures that the codebase can scale cleanly without intertwining business logic directly into active HTTP routing files.

2.1.1 Directory Breakdown

```
Backend
├── controllers
│   ├── groupController.js
│   ├── mediaController.js
│   └── routePlannerController.js
├── middleware
│   └── authMiddleware.js
├── node_modules
├── prisma
│   ├── migrations
│   └── schema.prisma
├── routes
│   ├── documentRoutes.js
│   ├── expenseRoutes.js
│   ├── groupRoutes.js
│   ├── mediaRoutes.js
│   ├── photoRoutes.js
│   ├── routePlannerRoutes.js
│   ├── settlementRoutes.js
│   └── userRoutes.js
├── services
│   ├── authService.js
│   ├── debtSimplification.js
│   ├── expenseService.js
│   ├── firebaseAdmin.js
```

```
│   ├── groupService.js
│   ├── mediaService.js
│   ├── memberInviteService.js
│   ├── orsService.js
│   ├── overpassService.js
│   ├── reportingService.js
│   ├── settlementService.js
│   └── tripService.js
├── tests
├── utils
│   ├── mediaStorage.js
│   ├── mediaUpload.js
│   ├── openingHoursParser.js
│   └── prismaClient.js
├── .env
├── .env.example
├── .gitignore
├── API_DOCUMENTATION.md
├── desc-mvp.md
├── package-lock.json
├── package.json
├── README.md
└── server.js
```

2.1.2 Directory Descriptions

- `controllers/`: Controllers handle incoming HTTP requests and coordinate the response flow. They usually validate request data, call the appropriate service logic, and return the final API response.
- `middleware/`: Middleware runs between the request and the final route handler. It is commonly used for tasks like authentication, logging, validation, and error handling.
- `node_modules/`: Contains all installed project dependencies and packages required for the application to run. It is automatically generated by npm/yarn and usually not edited manually.
- `prisma/`: Prisma is the ORM layer used to interact with the database in a structured and type-safe way. It manages schema definitions, database queries, and migrations.
- `routes/`: Routes define the API endpoints and map each request path to the correct controller or handler. They act as the entry points for different application features.
- `services/`: Services contain the core business logic of the application. They keep controllers lightweight by handling data processing, rules, and communication with the database or external systems.
- `tests/`: The tests folder contains automated test cases used to verify that the application works as expected. It helps catch bugs early and ensures reliability during changes.
- `utils/`: Utilities include reusable helper functions or common logic shared across different parts of the project. They reduce code duplication and improve maintainability.

2.1.3 Key Service Modules

Rather than monolithic design, logic is strictly segmented inside the `services` directory:

- `expenseService.js`: Operates algorithmic divisions ensuring accurate numeric representation for both completely "EQUAL" splits and mathematically distinct "CUSTOM" splits.
- `settlementService.js` & `debtSimplification.js`: The heaviest logical footprint, dictating live peer-to-peer balance tracking and actively implementing complex max-flow bidirectional netting algorithms to minimize actual transaction volume seamlessly.
- `groupService.js`: Manages trip initialization workflows, dynamic base-currency assignments leveraging IP geolocation, and explicit member invitation token creation.
- `planningService.js` & `orsService.js`: Reaches out to external OpenRouteService APIs, utilizing nearest-neighbor heuristics functionally enriching sequences of user-defined locations with accurate geographical mapping coordinates and operational timing limits.

2.2 Frontend Architecture

To combat unmanageable file bloat inherent in large centralized applications, the frontend repository completely abandons strictly technical folder structures (e.g., grouping all pure "UI" pieces together). Instead, it adopts a **Feature-First Architecture**, cleanly grouping codebases fundamentally by exactly what functional domain they impact.

2.2.1 Directory Breakdown

```
frontend
├── .dart_tool
├── .github
├── .idea
├── .vscode
├── android
├── assets
├── build
├── ios
├── lib
│   ├── app
│   │   ├── app_providers.dart
│   │   ├── app.dart
│   │   ├── main_screen.dart
│   │   └── routes.dart
│   ├── core
│   │   ├── api
│   │   ├── constants
│   │   ├── services
│   │   ├── theme
│   │   ├── utils
│   │   └── widgets
│   ├── features
│   │   ├── account_settings
│   │   ├── auth
│   │   ├── dashboard
│   │   ├── documents
│   │   │   ├── data
│   │   │   ├── presentation
│   │   │   └── documents_integration.md
│   │   ├── gallery
│   │   │   ├── data
│   │   │   │   ├── models
│   │   │   │   ├── repositories
│   │   │   │   └── services
│   │   │   ├── presentation
│   │   │   │   ├── providers
│   │   │   │   ├── screens
│   │   │   │   └── widgets
│   │   │   └── gallery_integration.md
```

```
│   │   ├── intro
│   │   ├── payments
│   │   ├── plan
│   │   │   ├── data
│   │   │   └── presentation
│   │   ├── trip_settings
│   │   └── trips
│   └── firebase_options.dart
├── linux
├── macos
├── test
├── web
├── windows
├── .env
├── .env.example
├── .flutter-plugins-dependencies
├── .gitignore
├── .metadata
├── analysis_options.yaml
├── analysis_output.txt
├── API_DOCUMENTATION.md
├── devtools_options.yaml
├── firebase.json
├── fix_members.py
├── flutter_01.png
├── frontend.iml
├── gallery_integration.md
├── git_status.txt
├── pubspec.lock
├── pubspec.yaml
└── README.md
```

2.2.2 Key Feature Modules

Each directory inside **features/** attempts strictly to encapsulate everything required to functionally operate that feature from top to bottom (its dedicated data models, local graphical widgets, and specific UI screens).

- **auth/**: Orchestrates completely secure user Firebase authentication, integrating initial Sign-Up and explicit Login workflows seamlessly returning necessary JWT tokens.
- **dashboard/**: Creates the central visual UI enumerating all active trip data dynamically.
- **trips/ & trip_settings/**: Comprehensive interfaces allowing rapid trip parameter creation, editing historical configurations, and orchestrating specific user invitation workflows.
- **payments/**: Represents the most intricate UI segment, cleanly rendering chronological group expenses, visualizing simplified debt visualizations, and creating native hooks invoking external UPI settlement functionality.
- **gallery/ & documents/**: Integrates specialized Flutter file/image pickers, cleanly pushing structured multipart data to the backend, and creating visual lists explicitly categorizing photos against PDF itineraries.

- **plan**/: Visually interfaces the backend's routing systems, creating sequenced graphic layouts logically detailing physical travel constraints and geographical durations.

2.3 Database

2.3.1 Structure

We hosted our Database on Neon. Here's the detailed structure:

```
├── Group/
│   ├── id
│   ├── title
│   ├── inviteLink
│   ├── createdBy
│   ├── currency
│   ├── createdAt
│   ├── simplifyDebts
│   ├── inviteLinkStatus
│   ├── preAddedParticipants
│   ├── destination
│   ├── startDate
│   ├── endDate
│   ├── tripType
│   ├── coverImage
│   ├── photoPath
│   └── photoUrl
├── User/
│   ├── id
│   ├── email
│   ├── passwordHash
│   ├── name
│   ├── upiId
│   ├── createdAt
│   ├── firebaseUid
│   └── phoneNumber
├── Transaction/
│   ├── id
│   ├── type
│   ├── groupId
│   ├── fromUserId
│   ├── toUserId
│   ├── amount
│   ├── currency
```



2.3.2 Key Fields

- **id:** id is the unique identifier assigned to a specific record or trip. It is used to distinguish and access that exact item in the system.
- **title**: title stores the name or heading of the trip or group. It helps users easily identify the record in the application.
- **inviteLink**: inviteLink contains the shareable link used to invite others to join the trip or group. It simplifies adding participants without manual entry.

- **createdBy**: createdBy stores the identifier of the user who created the trip or record. It helps track ownership and management responsibility.
- **currency**: currency defines the monetary unit used for expenses and settlements in the trip. It ensures all financial calculations follow the selected currency.
- **createdAt**: createdAt records the date and time when the record was created. It is useful for tracking history and sorting data chronologically.
- **simplifyDebts**: simplifyDebts indicates whether debt settlements should be reduced into fewer transactions. It makes repayments easier and less confusing for users.
- **inviteLinkStatus**: inviteLinkStatus shows whether the invitation link is active, inactive, or expired. It helps control whether new users can join through the link.
- **preAddedParticipants**: preAddedParticipants stores participants added in advance before they fully join the trip or group. It helps prepare the member list early for planning and expense tracking.
- **destination**: destination represents the place or target location of the trip. It helps define where the travel or event is intended to occur.
- **startDate**: startDate stores the date on which the trip or activity begins. It is used for scheduling and timeline management.
- **endDate**: endDate records the date on which the trip or activity finishes. It helps determine the overall duration of the trip.
- **tripType**: tripType specifies the category or nature of the trip, such as business, leisure, or group travel. It helps organize trips based on their purpose.
- **coverImage**: coverImage refers to the main image associated with the trip or group. It is typically used as a visual banner or display image in the interface.
- **photoPath**: photoPath stores the internal file path where an uploaded image is saved. It helps the system locate the image on storage.
- **photoUrl**: photoUrl contains the accessible URL of the uploaded or linked image. It is used to display the image in the frontend or share it externally.

2.4 System Environment Dependencies

At the root of both repositories lie heavily standard ecosystem configurations. The backend relies fundamentally on package.json to organize Node modules like multer, cors, and firebase-admin, while utilizing .env arrays strictly protecting secret keys. In tandem, the frontend exclusively utilizes pubspec.yaml logically coordinating structural packages exactly like provider, dio (for networking), and google_fonts directly controlling visual layouts across cross-compiled iOS and Android execution states.

3 Completeness

We were able to successfully implement most of the core functionalities described in the SRS. The system is built using a modular architecture with a Node.js + Express backend and a structured frontend, where each feature is handled through dedicated routes, controllers, services, and UI components.

3.1 User Authentication

3.1.1 User Registration / Login

- We have implemented user authentication using **Firebase Authentication**.
- Users can register and log in through the Flutter frontend using Firebase SDK.
- Name, email and phone number are stored in Neon DB. Passwords are handled entirely by Firebase, and are not stored manually in the backend.
- After successful login, Firebase generates an **ID token**, which is sent to the backend for verification.

3.1.2 Token Verification

- We have implemented authentication verification using Firebase Admin SDK in the backend (firebaseAdmin.js).
- Incoming requests include Firebase ID tokens, which are verified before processing.
- Middleware defined in middleware/authMiddleware.js is used to:
    - Validate tokens
    - Extract user identity
    - Attach user information to the request object
- This ensures that only authenticated users can access protected routes such as groups, expenses, and settlements.

3.1.3 Route Protection

- All protected APIs are secured using authentication middleware.
- Routes defined in files such as:
    - routes/groupRoutes.js
    - routes/expenseRoutes.js
    - routes/settlementRoutes.js
      require valid Firebase tokens before access is granted.
- Unauthorized or invalid requests are rejected with appropriate error responses.

3.2 User Profile

3.2.1 View Profile

- We have implemented “VIEW PROFILE” functionality using routes defined in routes/userRoutes.js.

- The request is handled in controllers/userController.js.
- User-related data (such as profile information) is stored in PostgreSQL and accessed using Prisma ORM.
- The frontend (Flutter) fetches profile data via API calls and displays it dynamically.

3.2.2 Update Profile

- We have implemented “EDIT PROFILE” functionality through API endpoints handled in userController.js.
- Update logic is implemented in the service layer, ensuring proper validation before updating the database.
- Changes are persisted using Prisma client (utils/prismaClient.js).
- Updated data is reflected across all modules since user identity is consistent with Firebase authentication.

3.3 Dashboard / Main Interface

3.3.1 Dashboard Access

- After successful authentication through Firebase, users are redirected to the main dashboard in the Flutter application.
- The dashboard acts as the central interface to access all features.

3.3.2 Navigation System

- Navigation between different modules is implemented using Flutter’s routing system.
- The dashboard provides access to:
    - Group Management
    - Expense Tracking
    - Settlements
    - Analysis
    - Media / Documents
- Each feature corresponds to backend APIs defined in modular route files.

3.3.3 Data Integration

- The frontend communicates with backend APIs using HTTP/Dio requests.
- Backend controllers process requests and return structured JSON responses.
- Data is dynamically updated and reflected on the dashboard without requiring manual refresh.

3.4 Expense Management

3.4.1 Add Expense

- We have implemented “ADD EXPENSE” using routes defined in routes/expenseRoutes.js.
- Requests are handled in controllers/expenseController.js.

- Core logic is implemented in services/expenseService.js.
- Expense data (amount, participants, split logic) is stored in PostgreSQL using Prisma ORM.
- Input validation ensures correctness of expense data before insertion.

3.4.2 View Expenses

- We have implemented “VIEW EXPENSES” functionality.
- Data retrieval is handled through controllers calling service-layer functions.
- Prisma queries fetch expenses associated with users and groups.
- The Flutter frontend displays this data dynamically.

3.4.3 Update Expense

- We have implemented “EDIT EXPENSE” using API endpoints in expenseRoutes.js.
- Update logic is handled in expenseController.js and expenseService.js.
- The system ensures that updates maintain consistency across shared expense data.

3.4.4 Delete Expense

- We have implemented “DELETE EXPENSE” using backend routes.
- Deletion logic is handled in the service layer.
- Authorization checks ensure that only permitted users can delete expenses.
- Changes are reflected immediately in the database and frontend.

3.4.5 Expense Splitting Logic

- Expense splitting is implemented in services/expenseService.js.
- The system calculates individual shares based on participants.
- These calculated values are stored and later used in settlement computations.
- This ensures accurate tracking of financial responsibilities among users.

3.5 Group Management

3.5.1 Create Group

- We have implemented “CREATE GROUP” functionality using backend routes defined in routes/groupRoutes.js.
- Requests are handled in controllers/groupController.js, where input validation and request processing are performed.
- Core group creation logic is implemented in services/groupService.js.
- Group data (group name, creator, members) is stored in PostgreSQL using Prisma ORM (prisma/schema.prisma).
- The authenticated user (verified via Firebase token) is automatically set as the group creator.

3.5.2 Manage Members

- We have implemented member management using APIs defined in groupRoutes.js.
- Member addition and invitation logic is handled in services/memberInviteService.js.

- Controllers validate requests and ensure only authorized users can modify group membership.
- Firebase-authenticated user identity is used to enforce access control.
- Database updates are performed using Prisma client (utils/prismaClient.js).

3.5.3 View Groups

- We have implemented “VIEW GROUPS” functionality where users can see all groups they belong to.
- Data is fetched through groupController.js and processed in groupService.js.
- Prisma queries retrieve group and member relationships from the database.
- The Flutter frontend fetches this data via API calls and displays it dynamically.

3.6 Settlements

3.6.1 Calculate Settlements

- We have implemented settlement calculation using APIs defined in routes/settlementRoutes.js.
- Core logic is handled in services/settlementService.js.
- A debt simplification algorithm is implemented in utils/debtSimplification.js to minimize the number of transactions.
- The algorithm computes optimized payment flows based on user balances derived from expenses.
- Only authenticated users (verified via Firebase tokens) can access settlement data.

3.6.2 Record Settlements

- We have implemented settlement recording using endpoints handled in controllers/settlementController.js.
- Settlement updates are processed in settlementService.js.
- Prisma ORM is used to update balances and persist settlement records in PostgreSQL.
- The system ensures consistency between expense data and settlement state.

3.6.3 View Settlement History

- We have implemented “VIEW SETTLEMENT HISTORY” functionality.
- Data is retrieved via settlementController.js and processed using service-layer functions.
- The Flutter frontend displays settlement history in a structured format.
- Only authorized users can access their settlement records.

3.7 Transactions / Financial Tracking

- We have implemented financial tracking using services defined in:
    - services/expenseService.js
    - services/settlementService.js

- All financial activities (expenses and settlements) are stored in PostgreSQL using Prisma ORM.
- Each transaction includes:
    - User/group association
    - Amount
    - Timestamp
- Backend services ensure:
    - Expense creation updates balances
    - Settlement updates reduce pending amounts
- Firebase-authenticated user identity ensures secure and user-specific financial tracking.
- The Flutter frontend fetches transaction data and displays it dynamically.

3.8 Analysis

- We have implemented analysis features using:
    - services/reportingService.js
    - services/planningService.js
- These services aggregate data from expenses, groups, and settlements.
- Backend processing includes:
    - Individual spending statistics
    - Group expense summaries
    - Trend-based insights
- The Flutter frontend displays this data using structured UI components.
- External integrations such as:
    - services/orsService.js
    - services/overpassService.js

      are used for route planning and location-based insights.
- This enhances analytical capabilities beyond basic expense tracking.

3.9 Media Management (Photo & Document Manager)

3.9.1 Upload Media (Photos & Documents)

- We have implemented media upload functionality using endpoints defined in routes/mediaRoutes.js.
- File handling is performed using multer middleware configured in utils/mediaUpload.js.
- Requests are processed in controllers/mediaController.js.
- Both images (photos) and documents (PDFs, files) can be uploaded through the Flutter frontend.
- Uploaded files are stored on the server filesystem.
- Metadata such as file name, type, associated group/user, and timestamps are stored in PostgreSQL using Prisma ORM.
- Firebase authentication ensures that only authorized users can upload media.

3.9.2 View Media (Gallery & Documents)

- We have implemented “VIEW MEDIA” functionality for both photos and documents.
- Data retrieval is handled via mediaController.js and processed in services/mediaService.js.
- Prisma queries fetch media records associated with users or groups.
- The Flutter frontend:
    - Displays images in a gallery format
    - Displays documents in a structured list with preview support
- Efficient rendering and caching improve performance.

3.9.3 Access / Download Media

- Users can access or download uploaded files through secured API endpoints.
- Backend serves files directly from the filesystem.
- Access control ensures that only authorized users (group members) can view or download media.

3.9.4 Delete Media

- We have implemented “DELETE MEDIA” functionality using secured routes.
- Requests are handled in mediaController.js and processed in mediaService.js.
- Files are removed from:
    - Server filesystem
    - Database records (via Prisma ORM)
- Authorization checks ensure only permitted users can delete media.

3.9.5 Media Security & Integration

- Firebase authentication is used to validate user identity for all media operations.
- File upload is securely handled using multer with controlled file handling.
- Metadata storage in PostgreSQL ensures structured access and future scalability.
- The system is designed to allow future integration with cloud storage services if required.

3.10 Future Implementations

Route Optimization with Place Recommendations
The system can be enhanced by integrating advanced route optimization algorithms along with intelligent place recommendation features. This would allow users to not only plan efficient travel routes but also receive suggestions for nearby points of interest such as restaurants, attractions, and accommodations based on their preferences and trip context.

Enhanced Search Functionality
The current search capabilities can be further improved by implementing a more robust and user-friendly search system. This includes features such as auto-suggestions, filtering based on categories (e.g., places, groups, expenses), and faster query handling to improve user experience while locating specific data or locations.

Multiple Currency Support and Conversion
The application can be extended to support multiple currencies, allowing users from different regions to manage expenses in their local currency. Integration of real-time currency conversion APIs would enable automatic conversion between currencies, ensuring accurate financial tracking across international users.

Scalable Media Storage Integration

Currently, media files are stored on the server filesystem. This can be improved by integrating cloud-based storage solutions such as AWS S3 or Firebase Storage to enhance scalability, reliability, and accessibility of media content.

Appendix A - Group Log

Date	Timing	Duration	Work Done
March 3	21:00-23:00	2 hrs	• Discussed the basic framework of the documents, the components to be present and divided it individually amongst ourselves
March 5	21:00-22:00	1 hr	• Decided database schema as well as backend technology to use
March 10	20:00-23:00	3 hrs	• Built individual components • Discussed on how to properly combine those components
March 12	21:00-23:30	2.5 hrs	• Revisions of the components and finalisation of frontend
March 15	20:00-01:00	5 hrs	• Working on backend
March 20	22:00-01:00	3 hrs	• Completion of frontend • Work on backend and its integration
March 23	20:00-21:30	1.5 hrs	• Started work on the Implementation Document and split up the work • Discussed minor changes to the application
March 25	21:00-00:00	3 hrs	• Completion of the Implementation Document • Completion of application
March 27	11:00-12:00	1 hr	• Final Proofread of Implementation Document